

因为解引用运算符的优先级低于点运算符，所以执行解引用运算的子表达式两端必须加上括号。如果没加括号，代码的含义就大不相同了：

```
// 运行 p 的 size 成员，然后解引用 size 的结果
*p.size();           // 错误：p 是一个指针，它没有名为 size 的成员
```

这条表达式试图访问对象 *p* 的 *size* 成员，但是 *p* 本身是一个指针且不包含任何成员，所以上述语句无法通过编译。

箭头运算符作用于一个指针类型的运算对象，结果是一个左值。点运算符分成两种情况：如果成员所属的对象是左值，那么结果是左值；反之，如果成员所属的对象是右值，那么结果是右值。

4.6 节练习

练习 4.20: 假设 *iter* 的类型是 `vector<string>::iterator`，说明下面的表达式是否合法。如果合法，表达式的含义是什么？如果不合法，错在何处？

(a) <code>*iter++;</code>	(b) <code>(*iter)++;</code>	(c) <code>*iter.empty();</code>
(d) <code>iter->empty();</code>	(e) <code>++*iter;</code>	(f) <code>iter++->empty();</code>

151

4.7 条件运算符

条件运算符 (`?:`) 允许我们把简单的 `if-else` 逻辑嵌入到单个表达式当中，条件运算符按照如下形式使用：

```
cond ? expr1 : expr2;
```

其中 *cond* 是判断条件的表达式，而 *expr1* 和 *expr2* 是两个类型相同或可能转换为某个公共类型的表达式。条件运算符的执行过程是：首先求 *cond* 的值，如果条件为真对 *expr1* 求值并返回该值，否则对 *expr2* 求值并返回该值。举个例子，我们可以使用条件运算符判断成绩是否合格：

```
string finalgrade = (grade < 60) ? "fail" : "pass";
```

条件部分判断成绩是否小于 60。如果小于，表达式的结果是 "fail"，否则结果是 "pass"。有点类似于逻辑与运算符和逻辑或运算符 (`&&` 和 `||`)，条件运算符只对 *expr1* 和 *expr2* 中的一个求值。

当条件运算符的两个表达式都是左值或者能转换成同一种左值类型时，运算的结果是左值；否则运算的结果是右值。

嵌套条件运算符

允许在条件运算符的内部嵌套另外一个条件运算符。也就是说，条件表达式可以作为另外一个条件运算符的 *cond* 或 *expr*。举个例子，使用一对嵌套的条件运算符可以将成绩分成三档：优秀 (high pass)、合格 (pass) 和不合格 (fail)：

```
finalgrade = (grade > 90) ? "high pass"
              : (grade < 60) ? "fail" : "pass";
```

第一个条件检查成绩是否在 90 分以上，如果是，执行符号?后面的表达式，得到 "high pass"；如果否，执行符号:后面的分支。这个分支本身又是一个条件表达式，它检查成绩是否在 60 分以下，如果是，得到 "fail"；否则得到 "pass"。

条件运算符满足右结合律，意味着运算对象（一般）按照从右向左的顺序组合。因此上面的代码中，靠右边的条件运算（比较成绩是否小于 60）构成了靠左边的条件运算的分支。



随着条件运算嵌套层数的增加，代码的可读性急剧下降。因此，条件运算的嵌套最好别超过两到三层。

在输出表达式中使用条件运算符

条件运算符的优先级非常低，因此当一条长表达式中嵌套了条件运算符表达式时，通常需要在它两端加上括号。例如，有时需要根据条件值输出两个对象中的一个，如果写这条语句时没把括号写全就有可能产生意想不到的结果：

152

```
cout << ((grade < 60) ? "fail" : "pass"); // 输出 pass 或者 fail
cout << (grade < 60) ? "fail" : "pass"; // 输出 1 或者 0!
cout << grade < 60 ? "fail" : "pass"; // 错误：试图比较 cout 和 60
```

在第二条表达式中，grade 和 60 的比较结果是 << 运算符的运算对象，因此如果 grade < 60 为真输出 1，否则输出 0。<< 运算符的返回值是 cout，接下来 cout 作为条件运算符的条件。也就是说，第二条表达式等价于

```
cout << (grade < 60); // 输出 1 或者 0
cout ? "fail" : "pass"; // 根据 cout 的值是 true 还是 false 产生对应的字面值
```

因为第三条表达式等价于下面的语句，所以它是错误的：

```
cout << grade; // 小于运算符的优先级低于移位运算符，所以先输出 grade
cout < 60 ? "fail" : "pass"; // 然后比较 cout 和 60!
```

4.7 节练习

练习 4.21: 编写一段程序，使用条件运算符从 `vector<int>` 中找到哪些元素的值是奇数，然后将这些奇数值翻倍。

练习 4.22: 本节的示例程序将成绩划分成 high pass、pass 和 fail 三种，扩展该程序使其进一步将 60 分到 75 分之间的成绩设定为 low pass。要求程序包含两个版本：一个版本只使用条件运算符；另外一个版本使用 1 个或多个 if 语句。哪个版本的程序更容易理解呢？为什么？

练习 4.23: 因为运算符的优先级问题，下面这条表达式无法通过编译。根据 4.12 节中的表（第 147 页）指出它的问题在哪里？应该如何修改？

```
string s = "word";
string p1 = s + s[s.size() - 1] == 's' ? "" : "s";
```

练习 4.24: 本节的示例程序将成绩划分成 high pass、pass 和 fail 三种，它的依据是条件运算符满足右结合律。假如条件运算符满足的是左结合律，求值过程将是怎样的？

4.8 位运算符

位运算符作用于整数类型的运算对象，并把运算对象看成是二进制位的集合。位运算符提供检查和设置二进制位的功能，如 17.2 节（第 640 页）将要介绍的，一种名为 `bitset`